UQAC

Université du Québec
à Chicoutimi

Plan de projet

Système de vidéo surveillance

1.1

Christian Kevin Sawadogo

Jérémie Arno Soppewe Tchawou

2026-10-27

Historique des révisions

Date	Version	Description	Auteur
2026-10-20	1.0	Version initiale (gabarit)	Jean-Luc Cyr
2026-10-27	1.1	Plan de projet complété : vue d'ensemble, références, équipe, contrôle des versions, ressources, calendrier, budget, risques et annexe A	Christian Kevin Sawadogo, Jérémie Arno Soppewe Tchawou

Définitions

Terme	Définition
Arrière-plan (modèle d')	Image de référence de la scène sans intrus, mise à jour progressivement pour suivre les changements lents (éclairage, météo).
Détection de changement	Comparaison de l'image courante avec l'arrière-plan pour repérer les pixels qui ont changé de façon significative.
Instantané	Image JPEG unique obtenue de la caméra par une requête HTTP.
Fausse alarme	Alerte émise alors qu'aucune intrusion réelle n'a eu lieu (variation d'éclairage, bruit, végétation, horodatage de la caméra).
Période de refroidissement	Délai minimal entre deux alertes pour un même événement, afin d'éviter l'envoi répété de messages.
Backend de calcul	Implémentation interchangeable des opérations de traitement d'image (GPU par OpenCL ou CPU par numpy) derrière une même interface.
Noyau (kernel)	Fonction exécutée en parallèle sur le GPU, typiquement un fil d'exécution par pixel.

Abréviations/acronymes

Abré./Acro.	Définition
ADR	Architecture Decision Record (fiche de décision d'architecture)
API	Application Programming Interface (interface de programmation)
CPU / GPU	Processeur central / processeur graphique
GUI	Graphical User Interface (interface graphique)
OpenCL	Open Computing Language, norme ouverte du groupe Khronos pour le calcul parallèle sur GPU et sur CPU de différents fabricants
PoCL	Portable Computing Language, implémentation libre d'OpenCL, utilisable notamment sur le CPU
SDP	Structure de découpage de projet
SemVer	Semantic Versioning (numérotation MAJEUR.MINEUR.CORRECTIF)
SMS / SMTP	Message texte / protocole d'envoi de courriel

Table des matières

Historique des révisions 2
Définitions 2
Abréviations/acronymes 2
Table des matières 3
1. Introduction 4
1.1. Vue d'ensemble du projet 4
1.2. Références 4
1.3. Portée 4
1.4. Livrables 5
2. Organisation du projet 6
2.1. Composition de l'équipe du projet 6
2.2. Stratégie de contrôle des versions 6
2.3. Directives de livraison 7
3. Ressources matérielles, calendrier et budget 7
3.1. Ressources matérielles 7
3.2. Calendrier des taches du projet 8
3.3. Budget 9
4. Gestion des risques 10
Annexe A – Directives de livraison 11

1. Introduction

1.1. Vue d'ensemble du projet

Le projet consiste à concevoir et réaliser un système de vidéosurveillance en Python. Le système récupère à intervalle régulier des images JPEG d'une caméra IP par HTTP, les analyse dans un processus de calcul séparé, accéléré par le GPU, pour détecter un changement ou un mouvement, puis envoie une alerte d'intrusion par courriel ou par texto.

Une interface graphique permet de configurer le système, de voir l'image courante avec les zones détectées et de suivre les alertes. OpenCV n'est pas utilisé : les algorithmes de traitement d'image sont réalisés par l'équipe avec numpy, Pillow et des noyaux GPU OpenCL.

L'architecture suit la vue globale de l'énoncé (figure 1) et comprend cinq composants : l'interface utilisateur (GUI), l'orchestrateur (MAIN), l'acquisition des images (CAMERA), le traitement d'image (CALCUL, processus séparé) et l'envoi des alertes (API de courriel ou de SMS).

1.2. Références

Les documents suivants ont servi à rédiger ce plan :

- Cyr, J.-L. (2026). Projet conception, 6GEI311 Architecture des logiciels, UQAC, automne 2026 (énoncé, version préliminaire).
- Gabarit « Plan de projet », version 1.0 du 2026-10-20, 6GEI311.
- ISO/IEC/IEEE 42010:2022, Software, systems and enterprise : Architecture description (vocabulaire du document d'architecture).
- ISO/IEC/IEEE 29148:2018, Systems and software engineering : Life cycle processes, Requirements engineering (structure des spécifications).
- ISO/IEC 25010:2023, Systems and software engineering : Systems and software Quality Requirements and Evaluation (SQuaRE), Product quality model (classement des exigences non fonctionnelles).
- IEC 62676-4:2025, Video surveillance systems for use in security applications, Part 4 : Application guidelines (lien entre analyse de risque et exigences).
- Documentation officielle de Python 3 (multiprocessing, tkinter), de PyOpenCL et de pytest.
- Recherche d'état de l'art de l'équipe : « Bâtir une vidéosurveillance Python sans OpenCV ni NVIDIA » (octobre 2026).

1.3. Portée

Concevoir et documenter l'architecture puis réaliser un logiciel mettant en commun des composants originaux et des composants de tiers.

- Communication réseau de haut niveau (appel web)
- Traitement d'images numériques (détection de changement/mouvement)
- Optimisation de calculs (pour le traitement de l'image)
- Rédaction de documents d'architecture logiciel

Ce système sera développé en Python . Le système devra récupérez des images à un intervalle régulier à partir d'une caméra en-ligne, les analysez pour détecter un changement et si c'est le cas, générez une alerte d'intrusion ou de mouvement.

Contrainte matérielle : aucun membre de l'équipe ne dispose d'une carte NVIDIA, ce qui exclut les bibliothèques propres à CUDA (Numba CUDA, CuPy). Le traitement GPU vise donc OpenCL sur le processeur graphique intégré (Intel ou AMD) des ordinateurs de l'équipe. La présence d'un pilote OpenCL fonctionnel sur chaque poste sera vérifiée dès la première semaine ; à défaut, le système bascule automatiquement sur une implémentation CPU de même interface.

Hors de la portée du projet : l'enregistrement vidéo continu, la reconnaissance d'objets ou de personnes, la gestion simultanée de plusieurs caméras et toute certification selon les normes de vidéosurveillance ou d'alarme.

1.4. Livrables

- Code source .py
- Fichier de dépendances pip à installer "requirements.txt"
- Configuration de git ".gitignore"
- Script d'initialisation de l'environnement tel que "setup.bat"
- Fichier d'instruction "readme.txt"
- Fichier de documentation d'architecture (en pdf)

Les instructions d'installation, configuration et exécutions doivent être fournies.

Les quatre documents de projets (Plan de projet, spécifications, composantes et architecture/conception).

Les instructions et le rapport doivent être déposés sur le site du cours ou dans le dépôt de source.

Échéancier des remises

Les livrables sont travaillés de manière asynchrone, en dehors des heures de laboratoire, et remis selon l'échéancier suivant. Le contenu de chaque remise correspond à l'interprétation de l'équipe des documents demandés ; il sera ajusté selon les précisions du chargé de projet.

Remise	Échéance	Contenu prévu
Plan de projet	27 octobre 2026, 23 h 59	Présent document : organisation, calendrier, budget, risques.
Ingénierie de concept	10 novembre 2026, 23 h 59	Spécifications : analyse des besoins, exigences fonctionnelles et non fonctionnelles vérifiables. Architecture conceptuelle : vue globale, choix technologiques justifiés (ADR). Preuves de concept : acquisition HTTP d'une image, exécution d'un noyau OpenCL.
Ingénierie préliminaire	24 novembre 2026, 23 h 59	Document des composantes : responsabilités, interfaces et dépendances de chaque module. Architecture préliminaire : vues logiques, processus et déploiement. Plan de tests et prototype v0.1.0 (acquisition, détection sur CPU, alerte par courriel).

Remise	Échéance	Contenu prévu
Ingénierie détaillée	16 décembre 2026, 23 h 59	Conception détaillée et document d'architecture final (PDF). Logiciel v1.0.0 : code source, requirements.txt, .gitignore, setup.bat, readme.txt, tests. Rapport : fonctionnement, architecture, algorithme, résultats et mesures de performance. Démonstration en laboratoire.

2. Organisation du projet

2.1. Composition de l'équipe du projet

L'équipe compte deux membres. Chaque membre est responsable de modules précis, et les livrables communs sont partagés.

Membre	Rôle	Responsabilités
Christian Kevin Sawadogo	Responsable acquisition et calcul ; coordination du projet	Module CAMERA : acquisition HTTP, gestion des erreurs et de la caméra hors ligne. Module CALCUL : algorithme de détection, processus séparé, backend GPU OpenCL et repli CPU. Banc d'essai de performance GPU et CPU. Suivi du calendrier et dépôt des remises.
Jérémie Arno Soppewe Tchawou	Responsable interface, alertes et qualité	Interface Tkinter (affichage, configuration, journal des alertes). Module d'alerte : courriel SMTP et SMS Twilio, période de refroidissement. Orchestrateur MAIN et fichier de configuration. Tests d'intégration et intégration continue.
Les deux membres	Responsabilités partagées	Analyse des besoins, document d'architecture et rapport final. Revue croisée de chaque demande de fusion. Validation du système et préparation de la démonstration.

2.2. Stratégie de contrôle des versions

- Outil : Git, avec un dépôt GitHub privé partagé avec l'auxiliaire et le chargé de projet.
- Flux de travail GitHub Flow : la branche main reste toujours fonctionnelle ; chaque fonctionnalité est développée dans une branche courte (par exemple feature/acquisition-http) puis intégrée par une demande de fusion relue par l'autre membre.
- Messages de commit au format Conventional Commits (feat, fix, docs, test, refactor).

- Identification des versions du logiciel selon SemVer, avec des étiquettes Git : v0.1.0 (prototype de l'ingénierie préliminaire, 24 novembre), v0.2.0 (GPU, interface et SMS intégrés) et v1.0.0 (ingénierie détaillée, 16 décembre).
- Identification des documents : numéro de version et historique des révisions dans chaque document ; chaque remise est marquée par une étiquette Git (par exemple remise-concept) ; sources (texte et diagrammes PlantUML ou Mermaid) versionnées dans le dossier docs/, PDF produits pour la remise.
- Dépendances : requirements.in (intention) et requirements.txt verrouillé avec pip-tools, compilé sous Windows ; une seule version de Python fixée dans setup.bat et identifiée dans le readme.
- Sécurité : identifiants SMTP et Twilio dans un fichier .env exclu par .gitignore ; un modèle .env.example est versionné.
- Intégration continue : GitHub Actions sur Windows exécute l'analyse statique (ruff) et les tests (pytest) à chaque demande de fusion.

2.3. Directives de livraison

Une démonstration du fonctionnement devra être faite en laboratoire devant l'auxiliaire du cours et/ou le chargé de projet en complément de la remise.

À remettre Tous les fichiers nécessaires pour mettre en œuvre l'application doivent être dans le dépôt de source et/ou le fichier zip.

Avant chaque remise, l'équipe vérifie l'installation complète avec setup.bat sur un poste propre, exécute la suite de tests et crée l'étiquette de version correspondante dans le dépôt. Une séquence d'images enregistrées est prévue pour faire la démonstration même si la caméra en ligne est indisponible.

3. Ressources matérielles, calendrier et budget

3.1. Ressources matérielles

Ressource	Description et utilisation
Deux ordinateurs portables Windows	Postes de développement personnels, GPU intégré Intel ou AMD (sans carte NVIDIA). Exécution des noyaux OpenCL ; la compatibilité OpenCL est vérifiée sur chaque poste en semaine 1.
Caméra IP publique Axis	Source d'images principale : http://205.237.248.39/axis-cgi/jpg/image.cgi?resolution=640x480
Caméra du local de laboratoire	Source de secours, sur demande à l'auxiliaire ou aux techniciens.
Séquences d'images enregistrées	Images de la caméra cible et extraits du jeu public CDnet : tests reproductibles, validation et démonstration de secours.

Ressource	Description et utilisation
Logiciels (gratuits)	Python 3 dans un environnement virtuel (version exacte fixée en semaine 1 selon la compatibilité de PyOpenCL), numpy, Pillow, scipy, PyOpenCL, Tkinter, requests, pytest, ruff, pip-tools ; Git et GitHub ; Word, draw.io ou PlantUML.
Services d'alerte	Gmail par SMTP avec mot de passe d'application, qui exige la validation en deux étapes (canal principal) ; compte d'essai Twilio pour les SMS (canal optionnel).

3.2. Calendrier des taches du projet

Le projet s'étend du 26 octobre au 16 décembre 2026. La structure de découpage du projet (SDP) suit les quatre remises : chaque période se termine par un jalon de remise à 23 h 59. Les phases de l'énoncé s'y répartissent ainsi : analyse des besoins et architecture conceptuelle pour l'ingénierie de concept ; architecture et composantes pour l'ingénierie préliminaire ; conception détaillée, réalisation, vérification et validation et livraison pour l'ingénierie détaillée. Les tests unitaires sont écrits en continu avec le code.

Semaine	Période	Tâches principales	Responsable	Livrable ou jalon
S1 : 26 oct. au 1er nov.	Plan de projet	Plan de projet. Création du dépôt et de setup.bat. Vérification d'OpenCL sur les deux postes.	Les deux	J1 : plan de projet (27 oct.)
S2 : 2 au 8 nov.	Ingénierie de concept	Analyse des besoins et exigences. Architecture conceptuelle et ADR. Preuves de concept (acquisition HTTP, noyau OpenCL).	Les deux ; preuves de concept : Christian	Brouillon des spécifications
S3 : 9 au 15 nov.	Concept, puis préliminaire	Révision croisée et remise du concept. Début du document des composantes et des interfaces.	Les deux	J2 : ingénierie de concept (10 nov.)
S4 : 16 au 22 nov.	Ingénierie préliminaire	Composantes, vues logiques, processus et déploiement. Prototype : acquisition, détection sur CPU, courriel. Plan de tests.	Christian (acquisition, détection) ; Jérémie (courriel, MAIN, plan de tests)	Prototype v0.1.0
S5 : 23 au 29 nov.	Préliminaire, puis détaillée	Remise de l'ingénierie préliminaire. Conception détaillée des modules.	Les deux	J3 : ingénierie préliminaire (24 nov.)
S6 : 30 nov. au 6 déc.	Ingénierie détaillée	Processus CALCUL et backend GPU OpenCL. Interface Tkinter et alerte SMS.	Christian (GPU) ; Jérémie (GUI, SMS)	v0.2.0

Semaine	Période	Tâches principales	Responsable	Livrable ou jalon
S7 : 7 au 13 déc.	Ingénierie détaillée	Intégration et tests d'intégration. Validation sur séquences enregistrées (précision, rappel, fausses alarmes par heure). Mesures de performance GPU et CPU. Rédaction du document d'architecture et du rapport.	Les deux	Rapport de validation
S8 : 14 au 16 déc.	Livraison	Installation sur poste propre, readme.txt. Révision finale et remise. Répétition de la démonstration.	Les deux	J4 : ingénierie détaillée v1.0.0 (16 déc.)

3.3. Budget

Le projet n'entraîne aucune dépense obligatoire. Le principal budget est le temps de l'équipe.

Poste	Estimation	Commentaire
Matériel	0 $	Ordinateurs personnels et caméras publiques ou du laboratoire.
Logiciels	0 $	Bibliothèques libres et gratuites.
Alertes par courriel	0 $	Compte Gmail existant.
Alertes par SMS (Twilio)	0 $ (essai) ; moins de 5 $ si payant	Le compte d'essai offre des crédits gratuits, mais n'envoie qu'à des numéros vérifiés et ajoute une mention d'essai aux messages. En compte payant, le tarif publié pour le Canada (octobre 2026, en dollars américains) est d'environ 1,15 $ par mois pour un numéro et de moins de 0,02 $ par SMS, frais d'opérateur compris.
Effort : planification et besoins	20 h	Plan de projet, spécifications.
Effort : architecture et conception	35 h	Composantes, ADR, conception détaillée.
Effort : réalisation	70 h	Code des cinq composants et tests unitaires.
Effort : vérification et validation	35 h	Tests d'intégration, validation, mesures de performance.
Effort : documentation et livraison	30 h	Document d'architecture, rapport, readme, démonstration.
Total de l'effort	190 h	Environ 95 h par membre sur huit semaines, soit environ 12 h par semaine par membre.

4. Gestion des risques

Risques principaux et stratégies pour les atténuer ou les contenir

Rappel : Probabilité, Conséquence, Exposition. La probabilité (P) et la conséquence (C) sont cotées de 1 (très faible) à 5 (très élevée). L'exposition (E) est le produit P × C (maximum 25). Les risques dont l'exposition est de 12 ou plus sont suivis chaque semaine.

#	Description	P	C	E	Méthodes de diminution ou contention
1	Aucune carte NVIDIA ; pilote OpenCL absent ou défectueux sur un poste	4	4	16	Backend OpenCL derrière une interface commune avec repli automatique sur numpy ; vérification d'OpenCL en semaine 1 ; le moteur utilisé est affiché et journalisé.
2	Retard dû à la charge de cours, aux examens de fin de session et au travail des membres	4	4	16	Prototype fonctionnel (CPU et courriel) dès le 24 novembre ; fonctionnalités terminées avant le 13 décembre ; suivi hebdomadaire de l'avancement.
3	Caméra en ligne indisponible le jour de la démonstration	3	5	15	Source d'images interchangeable : séquence enregistrée rejouée par un faux serveur caméra ; caméra du local en secours.
4	Pièges du multiprocessus sous Windows (démarrage spawn, interblocage des files)	4	3	12	Point d'entrée protégé, protocole d'arrêt par sentinelle, test d'intégration du processus CALCUL.
5	Interface Tkinter figée par le calcul ou le réseau	4	3	12	Aucun calcul dans le fil de l'interface ; lecture d'une file de résultats par root.after.
6	Différences d'environnement entre les deux postes	3	4	12	Version de Python unique, requirements.txt verrouillé, setup.bat testé sur les deux postes et par l'intégration continue.
7	Fausses alarmes trop nombreuses (éclairage, bruit, horodatage)	3	4	12	Moyenne mobile de l'arrière-plan, masque de région d'intérêt, aire minimale, confirmation sur plusieurs images, refroidissement ; mesure des fausses alarmes par heure.
8	Gain du GPU faible ou nul à 640x480, ce qui affaiblit la justification du choix	3	3	9	Mesures séparées du temps de calcul et du temps de bout en bout, à plusieurs résolutions ; résultats présentés honnêtement dans le rapport.
9	Limites du compte d'essai Twilio	3	3	9	Courriel comme canal principal ; essai réel des SMS en semaine 6.
10	Fuite d'identifiants dans le dépôt	2	4	8	Fichier .env exclu par .gitignore, modèle .env.example, revue des demandes de fusion.

Annexe A – Directives de livraison

Projet : Système de vidéosurveillance (6GEI311 Architecture des logiciels)

Client : Jean-Luc Cyr, ing., chargé de projet, UQAC

Date : 2026-10-27

Identification des livrables

Composant	Version livrée	Critère d'acceptation	Date de complétion
Plan de projet	1.1	Toutes les sections du gabarit complétées	27 oct. 2026
Ingénierie de concept : spécifications et architecture conceptuelle	1.0	Exigences identifiées, vérifiables, avec méthode de vérification ; choix technologiques justifiés	10 nov. 2026
Ingénierie préliminaire : composantes, architecture préliminaire, plan de tests	1.0	Chaque module décrit (responsabilité, interfaces, dépendances) ; prototype v0.1.0 fonctionnel	24 nov. 2026
Ingénierie détaillée : conception détaillée et document d'architecture (PDF)	1.0	Vues, diagrammes et décisions d'architecture justifiées	16 déc. 2026
Code source et dépôt GitHub	v1.0.0	Installation par setup.bat sur un poste propre ; tests pytest réussis	16 déc. 2026
requirements.txt, .gitignore, setup.bat, readme.txt	v1.0.0	Instructions d'installation, de configuration et d'exécution complètes	16 déc. 2026
Rapport	1.0	Fonctionnement, architecture, algorithme, résultats et mesures	16 déc. 2026
Démonstration en laboratoire	v1.0.0	Un mouvement est détecté et l'alerte est reçue devant l'auxiliaire	Date fixée par le cours

Approuvé par : Date d'approbation :